Guion de pruebas para el video — Avance 1 ISW-621

Este guion sigue, en orden, la sección 3 de las *Instrucciones de entrega*. Cada paso es una prueba que se ejecuta en Scalar y se muestra en el video.

Cómo leer cada paso

Todos los pasos tienen la misma estructura:

Línea	Qué significa
Endpoint	Método y ruta que se busca en la lista de Scalar.
Parámetros	Valores que se escriben en los campos de la ruta (`id`) o de la consulta (`status`, `assetId`…).
Respuesta esperada	Código HTTP que debe aparecer. Si sale otro, algo no está bien.
Qué mostrar	Lo que hay que señalar en la respuesta para que se vea en el video.
Qué decir	Frase corta para narrar. No se explica el código.
Cuerpo	El JSON que se copia completo y se pega sin cambiar nada.

Cómo ejecutar un paso en Scalar

1. Buscar el endpoint en la lista de la izquierda y abrirlo.
2. Presionar **Test Request**.
3. Escribir los **Parámetros** en sus campos.
4. Si el paso tiene cuerpo: en **Body** elegir **JSON** y pegar el bloque.
5. Presionar **Send** y hacer zoom a la respuesta (código y cuerpo).

Antes de grabar

- Recrear la base de datos (Parte 2). Los ids de este guion asumen una base nueva: los registros creados durante el video reciben el id **12** (activo y repuesto), **9** (categoría, técnico, especialidad y proveedor) y **11** (orden).
- Si se ensaya antes de grabar, hay que recrear la base otra vez para que los ids coincidan.
- Después de cada registro, actualización o asociación hay un paso **Comprobar…**: las instrucciones piden mostrar que el cambio quedó guardado.

Parte 1 — Presentación (30 segundos)

Decir el nombre del equipo, los integrantes (Andrey Delgado y Austin) y el contexto: **Equipos de TI / Data Center**. No se usan diapositivas ni se muestra código.

Parte 2 — Base de datos desde cero (1 minuto 30 segundos)

1. Mostrar en SQL Server Management Studio que la base `MantenimientoActivosDbDev` no existe.
2. En una terminal, en la carpeta de la solución, ejecutar:

```
dotnet ef database drop -f --project Proyecto.Infraestructure --startup-project Proyecto.WebAPI -- --
environment Development
dotnet ef database update --project Proyecto.Infraestructure --startup-project Proyecto.WebAPI -- --environment
Development
```

1. Mostrar la lista de migraciones aplicadas.
2. Iniciar el proyecto (F5) y mostrar en SQL Server que las tablas tienen datos, por ejemplo con `SELECT COUNT(*) FROM Assets;`. Repetir con Technicians, SpareParts y MaintenanceOrders: deben tener entre 8 y 12 filas.

Qué decir: "La base se crea desde cero con las migraciones y los datos iniciales se cargan solos."

Parte 3 — Documentación en Scalar (30 segundos)

1. Abrir `/scalar` y recorrer la lista de endpoints: activos, categorías, técnicos, especialidades, repuestos, proveedores, órdenes y catálogos.
2. Abrir un endpoint (por ejemplo, `GET /api/assets/{id}`) y mostrar el resumen, los códigos de respuesta y el esquema con los campos calculados.

Qué decir: "Scalar documenta todos los endpoints con sus códigos de respuesta."

Parte 4 — Gestión de Activos

Contrato 5.1 del enunciado: listar con filtros, detalle, registrar, actualizar, cambiar estado, dar de baja e imagen.

4.1 Listar todos los activos

- **Endpoint:** `GET /api/assets`
- **Parámetros:** ninguno
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Cada activo trae `categoryName`, `ageInMonths` (antigüedad en meses), `maintenanceOrderCount` e `imageUrl`. El activo dado de baja (PC-0001) no aparece.
- **Qué decir:** "Listar activos: responde 200 y muestra los campos calculados."

Sin cuerpo.

4.2 Filtrar activos por categoría

- **Endpoint:** `GET /api/assets`
- **Parámetros:** `categoryId` = 1
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Solo aparecen servidores (SRV-0001 y SRV-0002).
- **Qué decir:** "Filtro por categoría."

Sin cuerpo.

4.3 Filtrar activos por estado

- **Endpoint:** `GET /api/assets`
- **Parámetros:** `status` = OutOfService
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Solo aparece LAP-0002.
- **Qué decir:** "Filtro por estado."

Sin cuerpo.

4.4 Filtro con un estado que no existe

- **Endpoint:** `GET /api/assets`
- **Parámetros:** `status` = Roto
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** El error indica que el filtro status no es válido.
- **Qué decir:** "Filtro inválido: responde 400."

Sin cuerpo.

4.5 Detalle de un activo

- **Endpoint:** `GET /api/assets/{id}`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Todos los campos del activo, `category` con `preventiveThresholdHours`, `ageInMonths`, `maintenanceOrderCount` y la lista `maintenanceOrders` (código, fecha, tipo, prioridad y estado).
- **Qué decir:** "Detalle del activo con su categoría y sus órdenes."

Sin cuerpo.

4.6 Detalle de un activo que no existe

- **Endpoint:** `GET /api/assets/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que el activo no existe.
- **Qué decir:** "Activo inexistente: responde 404."

Sin cuerpo.

4.7 Registrar un activo

- **Endpoint:** `POST /api/assets`
- **Parámetros:** ninguno
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** El código lo asigna el sistema: `SRV-0003`. El id nuevo es 12.
- **Qué decir:** "Registrar un activo: responde 201 y el código lo genera el sistema."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Servidor de monitoreo",
  "description": "Ejecuta Zabbix para el data center",
  "brand": "Dell",
  "model": "PowerEdge R650",
  "serialNumber": "DL-R650-9QK44",
  "acquiredOn": "2025-06-10",
  "purchaseCost": 4200000,
  "usefulLifeMonths": 60,
  "usageHours": 2100,
  "criticality": "High",
  "categoryId": 1,
  "locationId": 1,
  "supplierId": 1
}
```

4.8 Registrar un activo con datos inválidos

- **Endpoint:** `POST /api/assets`
- **Parámetros:** ninguno
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** La respuesta lista cada campo que falló: `name`, `criticality` y `acquiredOn` (fecha futura).
- **Qué decir:** "Datos inválidos: responde 400 e indica cada campo."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "",
  "description": "Ejecuta Zabbix para el data center",
  "brand": "Dell",
  "model": "PowerEdge R650",
  "serialNumber": "DL-R650-9QK44",
  "acquiredOn": "2030-01-01",
  "purchaseCost": 4200000,
  "usefulLifeMonths": 60,
  "usageHours": 2100,
  "criticality": "Altisima",
  "categoryId": 1,
  "locationId": 1,
  "supplierId": 1
}
```

4.9 Registrar un activo con un número de serie repetido

- **Endpoint:** `POST /api/assets`
- **Parámetros:** ninguno
- **Respuesta esperada: 409 Conflict**
- **Qué mostrar:** El número de serie ya lo tiene SRV-0001.
- **Qué decir:** "El código del activo lo genera el sistema y nunca se repite; lo que se puede duplicar es el número de serie: responde 409."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Servidor de monitoreo",
  "description": "Ejecuta Zabbix para el data center",
  "brand": "Dell",
  "model": "PowerEdge R650",
  "serialNumber": "DL-R750-7HX21",
  "acquiredOn": "2025-06-10",
  "purchaseCost": 4200000,
  "usefulLifeMonths": 60,
  "usageHours": 2100,
  "criticality": "High",
  "categoryId": 1,
  "locationId": 1,
  "supplierId": 1
}
```

4.10 Comprobar que el activo quedó guardado

- **Endpoint:** `GET /api/assets/{id}`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Aparece el activo SRV-0003 con los datos enviados.
- **Qué decir:** "El registro quedó guardado."

Sin cuerpo.

4.11 Actualizar un activo

- **Endpoint:** `PUT /api/assets/{id}`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Nombre nuevo y `usageHours` en 2400.
- **Qué decir:** "Actualizar un activo: responde 200."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Servidor de monitoreo y alertas",
  "description": "Zabbix y Grafana para el data center",
  "brand": "Dell",
  "model": "PowerEdge R650",
  "serialNumber": "DL-R650-9QK44",
  "acquiredOn": "2025-06-10",
  "purchaseCost": 4200000,
  "usefulLifeMonths": 60,
  "usageHours": 2400,
  "criticality": "High",
  "categoryId": 1,
  "locationId": 1,
  "supplierId": 1
}
```

4.12 Actualizar un activo con datos inválidos

- **Endpoint:** `PUT /api/assets/{id}`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Error en `usefulLifeMonths`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Servidor de monitoreo y alertas",
  "description": "Zabbix y Grafana para el data center",
  "brand": "Dell",
  "model": "PowerEdge R650",
  "serialNumber": "DL-R650-9QK44",
  "acquiredOn": "2025-06-10",
  "purchaseCost": 4200000,
  "usefulLifeMonths": 0,
  "usageHours": 2400,
  "criticality": "High",
  "categoryId": 1,
  "locationId": 1,
  "supplierId": 1
}
```

4.13 Actualizar un activo que no existe

- **Endpoint:** `PUT /api/assets/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que el activo no existe.
- **Qué decir:** "Activo inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Servidor de monitoreo y alertas",
  "description": "Zabbix y Grafana para el data center",
  "brand": "Dell",
  "model": "PowerEdge R650",
  "serialNumber": "DL-R650-9QK44",
  "acquiredOn": "2025-06-10",
  "purchaseCost": 4200000,
  "usefulLifeMonths": 60,
  "usageHours": 2400,
  "criticality": "High",
  "categoryId": 1,
  "locationId": 1,
  "supplierId": 1
}
```

4.14 Comprobar la actualización

- **Endpoint:** `GET /api/assets/{id}`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Se ven el nombre y las horas nuevas.
- **Qué decir:** "La actualización quedó guardada."

Sin cuerpo.

4.15 Cambiar el estado a Fuera de Servicio

- **Endpoint:** `PATCH /api/assets/{id}/status`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** `status` = OutOfService.
- **Qué decir:** "Cambio de estado administrativo: responde 200."

Copiar y pegar en **Body → JSON**:

```
{
  "status": "OutOfService"
}
```

4.16 Cambiar a un estado que no existe

- **Endpoint:** `PATCH /api/assets/{id}/status`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Error en `status`.
- **Qué decir:** "Estado inválido: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "status": "Roto"
}
```

4.17 Cambiar el estado de un activo que no existe

- **Endpoint:** `PATCH /api/assets/{id}/status`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que el activo no existe.
- **Qué decir:** "Activo inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "status": "OutOfService"
}
```

4.18 Cambiar el estado de un activo con una orden abierta

- **Endpoint:** `PATCH /api/assets/{id}/status`
- **Parámetros:** `id` = 4
- **Respuesta esperada: 409 Conflict**
- **Qué mostrar:** NET-0002 tiene abierta la orden OT-2026-0003.
- **Qué decir:** "Tiene una orden abierta: responde 409."

Copiar y pegar en **Body → JSON**:

```
{
  "status": "OutOfService"
}
```

4.19 Comprobar el cambio de estado

- **Endpoint:** `GET /api/assets/{id}`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** `status` = OutOfService.
- **Qué decir:** "El estado quedó guardado."

Sin cuerpo.

4.20 Dar de baja un activo

- **Endpoint:** `DELETE /api/assets/{id}`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** El activo pasa a Decommissioned. No se borra de la base de datos.
- **Qué decir:** "Baja lógica: responde 200."

Sin cuerpo.

4.21 Dar de baja un activo con una orden abierta

- **Endpoint:** `DELETE /api/assets/{id}`
- **Parámetros:** `id` = 2
- **Respuesta esperada: 409 Conflict**
- **Qué mostrar:** SRV-0002 tiene una orden en ejecución.
- **Qué decir:** "Tiene una orden abierta: responde 409."

Sin cuerpo.

4.22 Dar de baja un activo que no existe o que ya fue dado de baja

- **Endpoint:** `DELETE /api/assets/{id}`
- **Parámetros:** `id` = 999 (y después repetir con `id` = 12)
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe o ya está dado de baja.
- **Qué decir:** "Inexistente o ya dado de baja: responde 404."

Sin cuerpo.

4.23 Comprobar la baja

- **Endpoint:** `GET /api/assets`
- **Parámetros:** `status` = Decommissioned
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Aparecen el activo 12 y PC-0001.
- **Qué decir:** "La baja quedó guardada y el activo conserva su historial."

Sin cuerpo.

4.24 Cargar la imagen de un activo

- **Endpoint:** `POST /api/assets/{id}/image`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Devuelve el nombre del archivo guardado.
- **Qué decir:** "Cargar imagen: responde 200."

En **Body** elegir **Multipart Form**: campo `file`, tipo **File**, seleccionar un archivo .png o .jpg de menos de 2 MB.

4.25 Cargar un archivo que no es imagen

- **Endpoint:** `POST /api/assets/{id}/image`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Mensaje de formato no permitido.
- **Qué decir:** "Formato inválido: responde 400."

En **Body** elegir **Multipart Form**: campo `file`, tipo **File**, seleccionar un archivo .pdf.

4.26 Cargar la imagen de un activo que no existe

- **Endpoint:** `POST /api/assets/{id}/image`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que el activo no existe.
- **Qué decir:** "Activo inexistente: responde 404."

En **Body** elegir **Multipart Form**: campo `file`, tipo **File**, seleccionar un archivo .png o .jpg de menos de 2 MB.

4.27 Comprobar la imagen

- **Endpoint:** `GET /api/assets/{id}`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** `imageUrl` cambió. Abrir esa dirección en el navegador muestra la imagen.
- **Qué decir:** "La imagen quedó guardada."

Sin cuerpo.

Parte 5 — Categorías de Activo

Contrato 5.2: listar, detalle, registrar, actualizar y eliminar.

5.1 Listar categorías

- **Endpoint:** `GET /api/asset-categories`
- **Parámetros:** ninguno
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Cada categoría trae `assetCount`.
- **Qué decir:** "Listar categorías con la cantidad de activos."

Sin cuerpo.

5.2 Detalle de una categoría

- **Endpoint:** `GET /api/asset-categories/{id}`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** `preventiveThresholdHours`, `assetCount` y la lista `assets` (código, nombre y estado).
- **Qué decir:** "Detalle con sus activos."

Sin cuerpo.

5.3 Detalle de una categoría que no existe

- **Endpoint:** `GET /api/asset-categories/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Categoría inexistente: responde 404."

Sin cuerpo.

5.4 Registrar una categoría

- **Endpoint:** `POST /api/asset-categories`
- **Parámetros:** ninguno
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** El id nuevo es 9.
- **Qué decir:** "Registrar categoría: responde 201."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Videovigilancia",
  "description": "Cámaras IP y grabadores NVR",
  "codePrefix": "CAM",
  "preventiveThresholdHours": 3000
}
```

5.5 Registrar una categoría con datos inválidos

- **Endpoint:** `POST /api/asset-categories`
- **Parámetros:** ninguno
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Errores en `name`, `codePrefix` y `preventiveThresholdHours`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "",
  "description": "Prueba",
  "codePrefix": "cam1",
  "preventiveThresholdHours": 0
}
```

5.6 Registrar una categoría con un nombre repetido

- **Endpoint:** `POST /api/asset-categories`
- **Parámetros:** ninguno
- **Respuesta esperada: 409 Conflict**
- **Qué mostrar:** Ya existe la categoría Servidores.
- **Qué decir:** "Nombre repetido: responde 409."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Servidores",
  "description": "Cámaras IP y grabadores NVR",
  "codePrefix": "SRVX",
  "preventiveThresholdHours": 3000
}
```

5.7 Comprobar el registro

- **Endpoint:** `GET /api/asset-categories/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Aparece Videovigilancia.
- **Qué decir:** "La categoría quedó guardada."

Sin cuerpo.

5.8 Actualizar una categoría

- **Endpoint:** `PUT /api/asset-categories/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Nombre nuevo y umbral en 2500.
- **Qué decir:** "Actualizar categoría: responde 200."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Videovigilancia IP",
  "description": "Cámaras IP, NVR y switches PoE",
  "codePrefix": "CAM",
  "preventiveThresholdHours": 2500
}
```

5.9 Actualizar una categoría con datos inválidos

- **Endpoint:** `PUT /api/asset-categories/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Error en `preventiveThresholdHours`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Videovigilancia IP",
  "description": "Cámaras IP, NVR y switches PoE",
  "codePrefix": "CAM",
  "preventiveThresholdHours": -5
}
```

5.10 Actualizar una categoría que no existe

- **Endpoint:** `PUT /api/asset-categories/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Categoría inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Videovigilancia IP",
  "description": "Cámaras IP, NVR y switches PoE",
  "codePrefix": "CAM",
  "preventiveThresholdHours": 2500
}
```

5.11 Comprobar la actualización

- **Endpoint:** `GET /api/asset-categories/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Se ven los datos nuevos.
- **Qué decir:** "La actualización quedó guardada."

Sin cuerpo.

5.12 Eliminar una categoría con activos

- **Endpoint:** `DELETE /api/asset-categories/{id}`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 409 Conflict**
- **Qué mostrar:** La categoría Servidores tiene activos.
- **Qué decir:** "Tiene activos: responde 409."

Sin cuerpo.

5.13 Eliminar una categoría

- **Endpoint:** `DELETE /api/asset-categories/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Categoría eliminada.
- **Qué decir:** "Eliminar categoría sin activos: responde 200."

Sin cuerpo.

5.14 Eliminar una categoría que no existe

- **Endpoint:** `DELETE /api/asset-categories/{id}`
- **Parámetros:** `id` = 9 (otra vez)
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Ya no existe.
- **Qué decir:** "Categoría inexistente: responde 404."

Sin cuerpo.

Parte 6 — Técnicos y Especialidades

Contrato 5.3: técnicos, especialidades, asociar y desasociar.

6.1 Listar técnicos

- **Endpoint:** `GET /api/technicians`
- **Parámetros:** ninguno
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Cada técnico trae `specialtyCount`.
- **Qué decir:** "Listar técnicos con la cantidad de especialidades."

Sin cuerpo.

6.2 Filtrar técnicos disponibles por especialidad

- **Endpoint:** `GET /api/technicians`
- **Parámetros:** `specialtyId` = 1, `isAvailable` = true
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Aparece Laura Mora.
- **Qué decir:** "Técnicos disponibles con una especialidad."

Sin cuerpo.

6.3 Filtrar técnicos no disponibles

- **Endpoint:** `GET /api/technicians`
- **Parámetros:** `isAvailable` = false
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Aparecen Andrea Vargas y Ricardo Jiménez.
- **Qué decir:** "Filtro por disponibilidad."

Sin cuerpo.

6.4 Detalle de un técnico

- **Endpoint:** `GET /api/technicians/{id}`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Laura Mora con 3 especialidades: nivel, `certifiedSince` y `seniorityInMonths` (antigüedad en meses).
- **Qué decir:** "Detalle con sus especialidades y la antigüedad en cada una."

Sin cuerpo.

6.5 Detalle de un técnico que no existe

- **Endpoint:** `GET /api/technicians/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Técnico inexistente: responde 404."

Sin cuerpo.

6.6 Registrar un técnico

- **Endpoint:** `POST /api/technicians`
- **Parámetros:** ninguno
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** El id nuevo es 9.
- **Qué decir:** "Registrar técnico: responde 201."

Copiar y pegar en **Body → JSON**:

```
{
  "fullName": "Valeria Rojas Méndez",
  "identificationNumber": "1-1987-0321",
  "email": "valeria.rojas@empresa.example",
  "phone": "8810-1009",
  "hourlyRate": 14000,
  "isAvailable": true
}
```

6.7 Registrar un técnico con datos inválidos

- **Endpoint:** `POST /api/technicians`
- **Parámetros:** ninguno
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Errores en `email`, `phone` y `hourlyRate`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "fullName": "Valeria Rojas Méndez",
  "identificationNumber": "1-1987-0321",
  "email": "no-es-correo",
  "phone": "12",
  "hourlyRate": -1,
  "isAvailable": true
}
```

6.8 Comprobar el registro

- **Endpoint:** `GET /api/technicians/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Aparece Valeria Rojas.
- **Qué decir:** "El técnico quedó guardado."

Sin cuerpo.

6.9 Actualizar un técnico

- **Endpoint:** `PUT /api/technicians/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Tarifa 15500 y no disponible.
- **Qué decir:** "Actualizar técnico: responde 200."

Copiar y pegar en **Body → JSON**:

```
{
  "fullName": "Valeria Rojas Méndez",
  "identificationNumber": "1-1987-0321",
  "email": "valeria.rojas@empresa.example",
  "phone": "8810-1009",
  "hourlyRate": 15500,
  "isAvailable": false
}
```

6.10 Actualizar un técnico con datos inválidos

- **Endpoint:** `PUT /api/technicians/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Error en `fullName`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "fullName": "",
  "identificationNumber": "1-1987-0321",
  "email": "valeria.rojas@empresa.example",
  "phone": "8810-1009",
  "hourlyRate": 15500,
  "isAvailable": false
}
```

6.11 Actualizar un técnico que no existe

- **Endpoint:** `PUT /api/technicians/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Técnico inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "fullName": "Valeria Rojas Méndez",
  "identificationNumber": "1-1987-0321",
  "email": "valeria.rojas@empresa.example",
  "phone": "8810-1009",
  "hourlyRate": 15500,
  "isAvailable": false
}
```

6.12 Listar especialidades

- **Endpoint:** `GET /api/specialties`
- **Parámetros:** ninguno
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Cada especialidad trae `technicianCount`.
- **Qué decir:** "Listar especialidades."

Sin cuerpo.

6.13 Registrar una especialidad

- **Endpoint:** `POST /api/specialties`
- **Parámetros:** ninguno
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** El id nuevo es 9.
- **Qué decir:** "Registrar especialidad: responde 201."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Telefonía IP",
  "description": "Centrales VoIP y teléfonos IP"
}
```

6.14 Registrar una especialidad con datos inválidos

- **Endpoint:** `POST /api/specialties`
- **Parámetros:** ninguno
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Error en `name`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "",
  "description": "Sin nombre"
}
```

6.15 Detalle de una especialidad

- **Endpoint:** `GET /api/specialties/{id}`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Los técnicos que la tienen, con su nivel y disponibilidad.
- **Qué decir:** "Detalle con sus técnicos."

Sin cuerpo.

6.16 Detalle de una especialidad que no existe

- **Endpoint:** `GET /api/specialties/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Especialidad inexistente: responde 404."

Sin cuerpo.

6.17 Actualizar una especialidad

- **Endpoint:** `PUT /api/specialties/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Nombre nuevo.
- **Qué decir:** "Actualizar especialidad: responde 200."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Telefonía IP y videoconferencia",
  "description": "VoIP y salas de videoconferencia"
}
```

6.18 Actualizar una especialidad con datos inválidos

- **Endpoint:** `PUT /api/specialties/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Error en `name`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "ab",
  "description": "Nombre muy corto"
}
```

6.19 Actualizar una especialidad que no existe

- **Endpoint:** `PUT /api/specialties/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Especialidad inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Telefonía IP y videoconferencia",
  "description": "VoIP y salas de videoconferencia"
}
```

6.20 Asociar una especialidad a un técnico

- **Endpoint:** `POST /api/technicians/{id}/specialties/{specialtyId}`
- **Parámetros:** `id` = 9, `specialtyId` = 9
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** El técnico ahora tiene 1 especialidad.
- **Qué decir:** "Asociar especialidad: responde 201."

Copiar y pegar en **Body → JSON**:

```
{
  "certificationLevel": "Intermediate",
  "certifiedSince": "2024-03-15"
}
```

6.21 Asociar con datos inválidos

- **Endpoint:** `POST /api/technicians/{id}/specialties/{specialtyId}`
- **Parámetros:** `id` = 9, `specialtyId` = 9
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Errores en `certificationLevel` y `certifiedSince` (fecha futura).
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "certificationLevel": "Expert",
  "certifiedSince": "2030-01-01"
}
```

6.22 Asociar con un técnico que no existe

- **Endpoint:** `POST /api/technicians/{id}/specialties/{specialtyId}`
- **Parámetros:** `id` = 999, `specialtyId` = 9
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que el técnico no existe.
- **Qué decir:** "Técnico inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "certificationLevel": "Intermediate",
  "certifiedSince": "2024-03-15"
}
```

6.23 Asociar la misma especialidad otra vez

- **Endpoint:** `POST /api/technicians/{id}/specialties/{specialtyId}`
- **Parámetros:** `id` = 9, `specialtyId` = 9
- **Respuesta esperada: 409 Conflict**
- **Qué mostrar:** El técnico ya tiene esa especialidad.
- **Qué decir:** "Especialidad repetida: responde 409."

Copiar y pegar en **Body → JSON**:

```
{
  "certificationLevel": "Intermediate",
  "certifiedSince": "2024-03-15"
}
```

6.24 Comprobar la asociación

- **Endpoint:** `GET /api/technicians/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** `specialtyCount` = 1 y la especialidad con su antigüedad.
- **Qué decir:** "La asociación quedó guardada."

Sin cuerpo.

6.25 Desasociar una especialidad

- **Endpoint:** `DELETE /api/technicians/{id}/specialties/{specialtyId}`
- **Parámetros:** `id` = 9, `specialtyId` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** El técnico queda sin especialidades.
- **Qué decir:** "Desasociar especialidad: responde 200."

Sin cuerpo.

6.26 Desasociar una especialidad que no tiene

- **Endpoint:** `DELETE /api/technicians/{id}/specialties/{specialtyId}`
- **Parámetros:** `id` = 9, `specialtyId` = 9 (otra vez)
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** El técnico no tiene esa especialidad.
- **Qué decir:** "No la tiene: responde 404."

Sin cuerpo.

Parte 7 — Repuestos y Proveedores

Contrato 5.4: repuestos, stock bajo el mínimo, proveedores, asociar y actualizar condiciones.

7.1 Listar repuestos

- **Endpoint:** `GET /api/spare-parts`
- **Parámetros:** ninguno
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** `isBelowMinimum`, `shortage` (faltante) y `supplierCount`. REP-BAT-RBC tiene el stock igual al mínimo: faltante 0.
- **Qué decir:** "Listar repuestos con sus campos calculados."

Sin cuerpo.

7.2 Repuestos con stock bajo el mínimo

- **Endpoint:** `GET /api/spare-parts/below-minimum`
- **Parámetros:** ninguno
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** REP-SSD-960, REP-TON-89A y REP-LCD-840.
- **Qué decir:** "Repuestos por debajo del mínimo."

Sin cuerpo.

7.3 Detalle de un repuesto

- **Endpoint:** `GET /api/spare-parts/{id}`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Proveedores con `offeredPrice` y `estimatedLeadTimeDays`.
- **Qué decir:** "Detalle con sus proveedores."

Sin cuerpo.

7.4 Detalle de un repuesto que no existe

- **Endpoint:** `GET /api/spare-parts/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Repuesto inexistente: responde 404."

Sin cuerpo.

7.5 Registrar un repuesto

- **Endpoint:** `POST /api/spare-parts`
- **Parámetros:** ninguno
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** El id nuevo es 12. `isBelowMinimum` = true y `shortage` = 1.
- **Qué decir:** "Registrar repuesto: responde 201."

Copiar y pegar en **Body → JSON**:

```
{
  "code": "REP-NIC-25G",
  "name": "Tarjeta de red 25 GbE",
  "description": "Dos puertos SFP28 para servidores",
  "unitOfMeasure": "Unit",
  "currentStock": 1,
  "minimumStock": 2
}
```

7.6 Registrar un repuesto con stock negativo

- **Endpoint:** `POST /api/spare-parts`
- **Parámetros:** ninguno
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Errores en `currentStock` (no puede ser negativo) y `unitOfMeasure`.
- **Qué decir:** "Stock negativo: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "code": "REP-NIC-10G",
  "name": "Tarjeta de red 25 GbE",
  "description": "Dos puertos SFP28 para servidores",
  "unitOfMeasure": "Caja",
  "currentStock": -3,
  "minimumStock": 2
}
```

7.7 Registrar un repuesto con un código repetido

- **Endpoint:** `POST /api/spare-parts`
- **Parámetros:** ninguno
- **Respuesta esperada: 409 Conflict**
- **Qué mostrar:** Ya existe ese código.
- **Qué decir:** "Código repetido: responde 409."

Copiar y pegar en **Body → JSON**:

```
{
  "code": "REP-SSD-960",
  "name": "Tarjeta de red 25 GbE",
  "description": "Dos puertos SFP28 para servidores",
  "unitOfMeasure": "Unit",
  "currentStock": 1,
  "minimumStock": 2
}
```

7.8 Actualizar un repuesto

- **Endpoint:** `PUT /api/spare-parts/{id}`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Stock 5: `isBelowMinimum` = false y `shortage` = 0.
- **Qué decir:** "Actualizar repuesto: responde 200."

Copiar y pegar en **Body → JSON**:

```
{
  "code": "REP-NIC-25G",
  "name": "Tarjeta de red 25 GbE",
  "description": "Dos puertos SFP28 para servidores",
  "unitOfMeasure": "Unit",
  "currentStock": 5,
  "minimumStock": 2
}
```

7.9 Actualizar un repuesto con datos inválidos

- **Endpoint:** `PUT /api/spare-parts/{id}`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Error en `minimumStock`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "code": "REP-NIC-25G",
  "name": "Tarjeta de red 25 GbE",
  "description": "Dos puertos SFP28 para servidores",
  "unitOfMeasure": "Unit",
  "currentStock": 5,
  "minimumStock": -1
}
```

7.10 Actualizar un repuesto que no existe

- **Endpoint:** `PUT /api/spare-parts/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Repuesto inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "code": "REP-NIC-25G",
  "name": "Tarjeta de red 25 GbE",
  "description": "Dos puertos SFP28 para servidores",
  "unitOfMeasure": "Unit",
  "currentStock": 5,
  "minimumStock": 2
}
```

7.11 Listar proveedores

- **Endpoint:** `GET /api/suppliers`
- **Parámetros:** ninguno
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Lista de proveedores.
- **Qué decir:** "Listar proveedores."

Sin cuerpo.

7.12 Registrar un proveedor

- **Endpoint:** `POST /api/suppliers`
- **Parámetros:** ninguno
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** El id nuevo es 9.
- **Qué decir:** "Registrar proveedor: responde 201."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Componentes de Red Pacífico",
  "taxId": "3-101-456709",
  "contactName": "Gabriela Mora",
  "phone": "2661-9900",
  "email": "ventas@redpacifico.example",
  "address": "Puntarenas centro",
  "generalConditions": "Envío gratis en pedidos mayores a 200 000 colones"
}
```

7.13 Registrar un proveedor con datos inválidos

- **Endpoint:** `POST /api/suppliers`
- **Parámetros:** ninguno
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Errores en `taxId` y `phone`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Componentes de Red Pacífico",
  "taxId": "123",
  "contactName": "Gabriela Mora",
  "phone": "abc",
  "email": "ventas@redpacifico.example",
  "address": "Puntarenas centro",
  "generalConditions": "Envío gratis en pedidos mayores a 200 000 colones"
}
```

7.14 Detalle de un proveedor

- **Endpoint:** `GET /api/suppliers/{id}`
- **Parámetros:** `id` = 1
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** `spareParts` con código, nombre, precio y días de entrega.
- **Qué decir:** "Detalle con los repuestos que suministra."

Sin cuerpo.

7.15 Detalle de un proveedor que no existe

- **Endpoint:** `GET /api/suppliers/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Proveedor inexistente: responde 404."

Sin cuerpo.

7.16 Actualizar un proveedor

- **Endpoint:** `PUT /api/suppliers/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Contacto nuevo.
- **Qué decir:** "Actualizar proveedor: responde 200."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Componentes de Red Pacífico",
  "taxId": "3-101-456709",
  "contactName": "Gabriela Mora Solís",
  "phone": "2661-9900",
  "email": "ventas@redpacifico.example",
  "address": "Puntarenas centro",
  "generalConditions": "Envío gratis en pedidos mayores a 200 000 colones",
  "isActive": true
}
```

7.17 Actualizar un proveedor con datos inválidos

- **Endpoint:** `PUT /api/suppliers/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Error en `name`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "",
  "taxId": "3-101-456709",
  "contactName": "Gabriela Mora Solís",
  "phone": "2661-9900",
  "email": "ventas@redpacifico.example",
  "address": "Puntarenas centro",
  "generalConditions": "Envío gratis en pedidos mayores a 200 000 colones",
  "isActive": true
}
```

7.18 Actualizar un proveedor que no existe

- **Endpoint:** `PUT /api/suppliers/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Proveedor inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "name": "Componentes de Red Pacífico",
  "taxId": "3-101-456709",
  "contactName": "Gabriela Mora Solís",
  "phone": "2661-9900",
  "email": "ventas@redpacifico.example",
  "address": "Puntarenas centro",
  "generalConditions": "Envío gratis en pedidos mayores a 200 000 colones",
  "isActive": true
}
```

7.19 Asociar un proveedor a un repuesto

- **Endpoint:** `POST /api/spare-parts/{id}/suppliers/{supplierId}`
- **Parámetros:** `id` = 12, `supplierId` = 9
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** El repuesto ahora tiene 1 proveedor.
- **Qué decir:** "Asociar proveedor: responde 201."

Copiar y pegar en **Body → JSON**:

```
{
  "offeredPrice": 185000,
  "estimatedLeadTimeDays": 6
}
```

7.20 Asociar con datos inválidos

- **Endpoint:** `POST /api/spare-parts/{id}/suppliers/{supplierId}`
- **Parámetros:** `id` = 12, `supplierId` = 9
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Errores en `offeredPrice` y `estimatedLeadTimeDays`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "offeredPrice": -10,
  "estimatedLeadTimeDays": 500
}
```

7.21 Asociar con un repuesto que no existe

- **Endpoint:** `POST /api/spare-parts/{id}/suppliers/{supplierId}`
- **Parámetros:** `id` = 999, `supplierId` = 9
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Repuesto inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "offeredPrice": 185000,
  "estimatedLeadTimeDays": 6
}
```

7.22 Comprobar la asociación

- **Endpoint:** `GET /api/spare-parts/{id}`
- **Parámetros:** `id` = 12
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** `supplierCount` = 1 y el proveedor con su precio y días.
- **Qué decir:** "La asociación quedó guardada."

Sin cuerpo.

7.23 Actualizar las condiciones del proveedor

- **Endpoint:** `PUT /api/spare-parts/{id}/suppliers/{supplierId}`
- **Parámetros:** `id` = 12, `supplierId` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Precio 172500 y 4 días.
- **Qué decir:** "Actualizar condiciones: responde 200."

Copiar y pegar en **Body → JSON**:

```
{
  "offeredPrice": 172500,
  "estimatedLeadTimeDays": 4
}
```

7.24 Actualizar condiciones con datos inválidos

- **Endpoint:** `PUT /api/spare-parts/{id}/suppliers/{supplierId}`
- **Parámetros:** `id` = 12, `supplierId` = 9
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Error en `offeredPrice`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "offeredPrice": -1,
  "estimatedLeadTimeDays": 4
}
```

7.25 Actualizar condiciones de un proveedor no asociado

- **Endpoint:** `PUT /api/spare-parts/{id}/suppliers/{supplierId}`
- **Parámetros:** `id` = 12, `supplierId` = 1
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Ese proveedor no está asociado al repuesto.
- **Qué decir:** "No está asociado: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "offeredPrice": 172500,
  "estimatedLeadTimeDays": 4
}
```

7.26 Comprobar las condiciones

- **Endpoint:** `GET /api/suppliers/{id}`
- **Parámetros:** `id` = 9
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Aparece REP-NIC-25G con precio 172500 y 4 días.
- **Qué decir:** "Las condiciones quedaron guardadas."

Sin cuerpo.

Parte 8 — Órdenes de Mantenimiento

Contrato 5.5: crear una solicitud, listar con filtro por activo y ver el detalle.

8.1 Crear una solicitud de mantenimiento

- **Endpoint:** `POST /api/maintenance-orders`
- **Parámetros:** ninguno
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** `status` = Requested, `workOrderCode` = OT-2026-0011, `daysSinceRequest` = 0. El id nuevo es 11.
- **Qué decir:** "Crear solicitud: responde 201 y el estado siempre es Requested."

Copiar y pegar en **Body → JSON**:

```
{
  "assetId": 3,
  "maintenanceTypeId": 1,
  "supplierId": 2,
  "title": "Preventivo del switch core",
  "reason": "Se acerca al umbral de horas de uso de la categoría",
  "description": "Limpieza, revisión de ventiladores y respaldo de configuración",
  "priority": "Medium"
}
```

8.2 Crear una solicitud con datos inválidos

- **Endpoint:** `POST /api/maintenance-orders`
- **Parámetros:** ninguno
- **Respuesta esperada: 400 Bad Request**
- **Qué mostrar:** Errores en `title` y `priority`.
- **Qué decir:** "Datos inválidos: responde 400."

Copiar y pegar en **Body → JSON**:

```
{
  "assetId": 3,
  "maintenanceTypeId": 1,
  "supplierId": 2,
  "title": "",
  "reason": "Se acerca al umbral de horas de uso de la categoría",
  "description": "Limpieza, revisión de ventiladores y respaldo de configuración",
  "priority": "Altisima"
}
```

8.3 Crear una solicitud para un activo que no existe

- **Endpoint:** `POST /api/maintenance-orders`
- **Parámetros:** ninguno
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que el activo no existe.
- **Qué decir:** "Activo inexistente: responde 404."

Copiar y pegar en **Body → JSON**:

```
{
  "assetId": 999,
  "maintenanceTypeId": 1,
  "supplierId": 2,
  "title": "Preventivo del switch core",
  "reason": "Se acerca al umbral de horas de uso de la categoría",
  "description": "Limpieza, revisión de ventiladores y respaldo de configuración",
  "priority": "Medium"
}
```

8.4 Intentar crear la orden con otro estado

- **Endpoint:** `POST /api/maintenance-orders`
- **Parámetros:** ninguno
- **Respuesta esperada: 201 Created**
- **Qué mostrar:** Aunque se envía `status` = Completed, la orden se crea en Requested.
- **Qué decir:** "El cliente no puede elegir el estado: siempre queda en Requested."

Copiar y pegar en **Body → JSON**:

```
{
  "assetId": 3,
  "maintenanceTypeId": 1,
  "supplierId": 2,
  "title": "Prueba de estado forzado",
  "reason": "Se acerca al umbral de horas de uso de la categoría",
  "description": "Limpieza, revisión de ventiladores y respaldo de configuración",
  "priority": "Medium",
  "status": "Completed"
}
```

8.5 Listar órdenes

- **Endpoint:** `GET /api/maintenance-orders`
- **Parámetros:** ninguno
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Cada orden trae `assetCode`, `assetName` y `daysSinceRequest`.
- **Qué decir:** "Listar órdenes con sus campos calculados."

Sin cuerpo.

8.6 Listar órdenes de un activo

- **Endpoint:** `GET /api/maintenance-orders`
- **Parámetros:** `assetId` = 3
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Solo las órdenes de NET-0001.
- **Qué decir:** "Filtro por activo."

Sin cuerpo.

8.7 Listar órdenes de un activo que no existe

- **Endpoint:** `GET /api/maintenance-orders`
- **Parámetros:** `assetId` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que el activo no existe.
- **Qué decir:** "Activo inexistente: responde 404."

Sin cuerpo.

8.8 Detalle de una orden

- **Endpoint:** `GET /api/maintenance-orders/{id}`
- **Parámetros:** `id` = 11
- **Respuesta esperada: 200 OK**
- **Qué mostrar:** Todos los campos, `asset` (id, código, nombre, estado y categoría) y `daysSinceRequest`.
- **Qué decir:** "Detalle con el activo relacionado."

Sin cuerpo.

8.9 Detalle de una orden que no existe

- **Endpoint:** `GET /api/maintenance-orders/{id}`
- **Parámetros:** `id` = 999
- **Respuesta esperada: 404 Not Found**
- **Qué mostrar:** Mensaje de que no existe.
- **Qué decir:** "Orden inexistente: responde 404."

Sin cuerpo.

Parte 9 — Logging y manejo de errores (1 minuto)

Tener visible la consola donde corre la API.

1. Ejecutar `GET /api/assets/{id}` con `id` = 1. En la consola aparecen dos líneas **[INF]**: una al empezar y otra al terminar, con el id del activo.
2. Ejecutar `DELETE /api/asset-categories/{id}` con `id` = 1. Responde **409** y en la consola aparece una línea **[WRN]** con el motivo.
3. Copiar el valor de `traceId` de la respuesta del paso 2 y buscarlo en la consola (o en `Proyecto.WebAPI/Logs/log-AAAAMMDD.txt`). Es el mismo valor que aparece entre corchetes en la línea [WRN].

Qué decir: "Cada solicitud se registra en el log; el error de negocio queda como Warning con el mismo traceId de la respuesta."

Lista de entrega (antes del domingo 25 a las 12:00 m.d.)

- [ ] El proyecto compila y `dotnet test` pasa.
- [ ] Las migraciones se aplican sobre una base vacía (Parte 2).
- [ ] Commit final en GitLab con el tag: `git tag avance-1` y luego `git push origin avance-1`.
- [ ] Archivo .zip de ese mismo commit, sin las carpetas `bin/`, `obj/` ni `.vs/`.
- [ ] Video de 15 minutos como máximo, con audio, en YouTube (no listado) o en Drive con acceso por enlace. Probar el enlace en una ventana de incógnito.
- [ ] Tabla de verificación completa: hash del commit del tag, enlace y duración del video, minuto (mm:ss) de cada rubro y observaciones donde haga falta.